

THAPAR INSTITUTE OF ENGINEERING AND TECHNOLOGY

Department of Computer Science and Engineering

UCS503 – SOFTWARE ENGINEERING

BharatOS

A Unified Civic Data Discovery and Developer API Platform

SOFTWARE ENGINEERING LAB REPORT

Submitted by

1024240135	Shivansh Singh
1024240155	Palveen Kaur
1024240106	Rajuss Sachdev
1024240006	Shivam Chauhan

Group: To be confirmed

B.E. Third Year, Computer Science and Engineering
Academic Year 2026–27, Odd Semester

Faculty Advisor

To be confirmed

Computer Science and Engineering Department
Thapar Institute of Engineering and Technology, Patiala

Abstract

BharatOS is an academic civic-data platform intended to reduce the effort required to discover and integrate permitted Indian public information. Government information is distributed across multiple portals and APIs that may differ in field names, schemas, authentication methods, update schedules, provenance conventions, and availability characteristics. BharatOS introduces a common discovery and normalization layer while preserving source attribution and explicit system boundaries.

The project has evolved through two implementation stages. An earlier Python-first Phase 1 baseline demonstrated citizen-facing scheme discovery, scheme-detail pages, a developer console, API documentation, an administrator preview, typed API contracts, provenance, and UML-based system modelling. That version used FastAPI, Jinja, Pydantic, custom CSS, minimal JavaScript, static validated prototype records, a fixed demonstration key, and nineteen automated tests.

The current working prototype moves the developer-platform portion closer to the intended infrastructure architecture. It uses Node.js/Express, PostgreSQL, Docker Compose, real API-key generation, SHA-256 key hashing, Bearer authentication, a real Open Government Data Platform India PIN-code source, normalized civic-data responses, persistent usage logs, per-key analytics, and an in-memory rate limiter of 20 requests per minute. PostgreSQL is isolated within Docker networking, while the API is exposed locally on port 8001. The PIN-code endpoint has been successfully tested with live Department of Posts data, and rate-limit behaviour has been verified through repeated requests.

This report documents both the earlier design baseline and the current implementation, while clearly distinguishing implemented functionality from future work such as Redis-backed quotas, scheduled ingestion, broader dataset coverage, production HTTPS, and AWS deployment.

Keywords: civic technology, government data, REST API, normalization, provenance, Docker, PostgreSQL, API keys, rate limiting, software engineering.

Contents

Abstract	i
1 Introduction	1
1.1 Background and Context	1
1.2 Problem Statement	1
1.3 Project Objectives	2
1.4 Scope and Phase Boundary	2
2 Requirements Analysis	4
2.1 Stakeholders and Actors	4
2.2 Functional Requirements	4
2.3 Non-functional Requirements	5
2.4 Use Case Model	6
3 Methodology and System Design	7
3.1 Development Methodology	7
3.2 High-Level Architecture	8
3.3 Target Architecture from Earlier Design	9
3.4 Domain Model and Class Design	9
3.5 Protected API Request Flow	10
3.6 Ingestion Activity	10
3.7 Technical Stack Evolution	11
3.8 Standard API Contract	11
4 Implementation	13

4.1	Earlier Python-first Phase 1 Baseline	13
4.2	Current Backend File Constraint	13
4.3	API-Key Generation and Storage	14
4.4	PostgreSQL Schema	14
4.4.1	api_keys	14
4.4.2	usage_logs	14
4.5	Civic-Data Adapter	15
4.6	Usage Analytics	15
4.7	Rate Limiting	16
4.8	Upstream Rate-Limit Handling	16
4.9	Docker Compose Environment	17
5	Results and Evaluation	18
5.1	Testing Methodology	18
5.2	Measured Current Results	18
5.3	Rate-Limit Verification	19
5.4	Performance Observation	20
5.5	Objective Evaluation	20
5.6	Risks and Mitigations	20
6	Phased Roadmap	22
6.1	Completed Infrastructure Milestones	22
6.2	Next Phase – Developer Frontend	22
6.3	AWS Deployment	23
6.4	Redis and Caching	23
6.5	Dataset Expansion	23
6.6	Operations and Evaluation	24
7	Conclusion	25
7.1	Summary of Work	25
7.2	Key Findings	26
7.3	Future Work and Recommendations	26

7.4 Final Remarks	27
A Demonstration Script	28
B Requirements Traceability	29
C Current Prototype API Endpoints	30

Introduction

1.1 Background and Context

India publishes substantial civic and administrative information through ministry portals, the Open Government Data platform, API Setu, and other official systems. Availability alone does not guarantee developer accessibility. Sources may use different formats, field names, authentication methods, update schedules, and metadata conventions. Citizens can face a similar discovery problem when they must visit several portals to locate potentially relevant schemes or civic information.

BharatOS addresses this fragmentation through two complementary ideas:

1. a citizen-facing discovery layer for source-linked civic information; and
2. a developer-facing REST interface with predictable success and error envelopes.

Official publishers remain authoritative. BharatOS is not intended to replace government systems or claim official status. Its purpose is to act as an academic discovery, normalization, and developer-access layer over permitted public/open information.

1.2 Problem Statement

Developers and citizens can spend more time locating, interpreting, and reconciling government information than actually using it. Equivalent concepts may have different names, schemas, file formats, and freshness indicators.

Problem: How can permitted Indian public schemes and civic data be presented through a single, source-transparent discovery platform and a consistent developer API without implying official status or collecting unnecessary personal information?

The current infrastructure prototype narrows this broad problem to a concrete developer flow: obtain a BharatOS API key, request a six-digit PIN code, retrieve real public data from the Department of Posts, normalize the response, record usage, and enforce a per-key request quota.

1.3 Project Objectives

1. Build a developer-facing civic-data API with versioned endpoints.
2. Normalize heterogeneous upstream fields into a stable BharatOS schema.
3. Preserve source attribution and provenance in API responses.
4. Generate secure developer API keys and avoid plaintext key storage.
5. Reject missing, invalid, and revoked keys with standardized errors.
6. Persist API-key metadata and request logs in PostgreSQL.
7. Provide usage analytics per API key.
8. Implement rate limiting and return HTTP 429 when a quota is exceeded.
9. Containerize the backend and database using Docker Compose.
10. Prepare the system for a later developer frontend and AWS deployment.
11. Maintain explicit boundaries between prototype behaviour and production-ready features.

1.4 Scope and Phase Boundary

The earlier Python-first Phase 1 focused on interface validation, API-contract design, provenance, UML artefacts, and a deterministic localhost demonstration. It included eight scheme snapshots, three PIN-code records, a fixed demonstration key, and browser-level developer tooling.

The current prototype advances the developer-platform infrastructure beyond that baseline. It now includes PostgreSQL persistence, randomly generated API keys, hashed key storage, live OGD PIN-code integration, persistent request logging, per-key usage analytics, and per-key rate limiting.

The following are still outside the current implementation scope:

- production user registration and sessions;

- API-key revocation UI;
- Redis-backed distributed rate limiting and caching;
- scheduled ingestion and multi-source ETL;
- full citizen discovery portal migration into the current Node-based stack;
- production HTTPS/TLS;
- high-load benchmark claims;
- AWS production deployment;
- automated backup and recovery.

Requirements Analysis

2.1 Stakeholders and Actors

Actor	Role	Primary Interactions
Anonymous Public User	Uses discovery functionality without an account.	Search/filter schemes, inspect details, view provenance, and review possible relevance.
Registered Developer	Uses authenticated developer APIs.	Generate/manage keys, call APIs, test endpoints, inspect normalized JSON, and view usage.
Administrator	Operates the platform.	Manage sources, monitor freshness, review usage/abuse, and oversee ingestion.
Government Data Source	External publisher of permitted records.	Supplies public API responses or documented source data.
Scheduler	Future external time-based trigger.	Starts scheduled ingestion and refresh jobs.

2.2 Functional Requirements

ID	Requirement	Current Status
FR-01	Search schemes using composable filters.	Earlier baseline implemented
FR-02	Display source-linked summaries and provenance.	Earlier baseline implemented
FR-03	Return normalized civic data through versioned REST endpoints.	Implemented

FR-04	Reject missing/invalid API keys using standard errors.	Implemented
FR-05	Generate random developer API keys.	Implemented
FR-06	Store only secure API-key hashes.	Implemented
FR-07	Persist API usage logs.	Implemented
FR-08	Provide per-key usage analytics.	Implemented
FR-09	Enforce per-key rate limits.	Implemented
FR-10	Retrieve live PIN-code data from a permitted public source.	Implemented
FR-11	Support API-key revocation workflow.	Backend status field exists; workflow pending
FR-12	Ingest, validate, normalize, and deduplicate multiple civic datasets.	Future work
FR-13	Provide Redis-backed caching and distributed quota state.	Future work
FR-14	Deploy the containerized stack to AWS.	Planned

2.3 Non-functional Requirements

- **Usability:** developer workflows should be understandable from request/response structure and future UI.
- **Consistency:** success and error responses should use predictable envelopes and error codes.
- **Security:** API keys should not be stored in plaintext; protected routes require Bearer authentication.
- **Traceability:** civic-data responses should identify their source and platform.
- **Maintainability:** upstream provider logic should remain isolated behind an adapter module.
- **Reliability:** upstream failures and throttling should be converted into explicit BharatOS errors.
- **Portability:** the same Docker-oriented architecture should run locally and later on AWS.
- **Privacy:** the platform should avoid unnecessary personal information.

2.4 Use Case Model

The earlier report modelled Registered Developer as a specialization of Anonymous Public User because a developer can retain public discovery capabilities while gaining authenticated developer features. Mandatory reused behaviour included provenance display, key validation, quota checking, normalization, and deduplication.

In the current implementation, the developer-facing use cases that are concretely demonstrated are:

- generate API key;
- call protected API;
- receive 401 for missing/invalid keys;
- query normalized PIN-code data;
- view usage summary;
- trigger a 429 response when quota is exceeded.

Methodology and System Design

3.1 Development Methodology

The project follows incremental, risk-first development. Instead of implementing the entire long-term architecture at once, each stage is required to produce observable working behaviour.

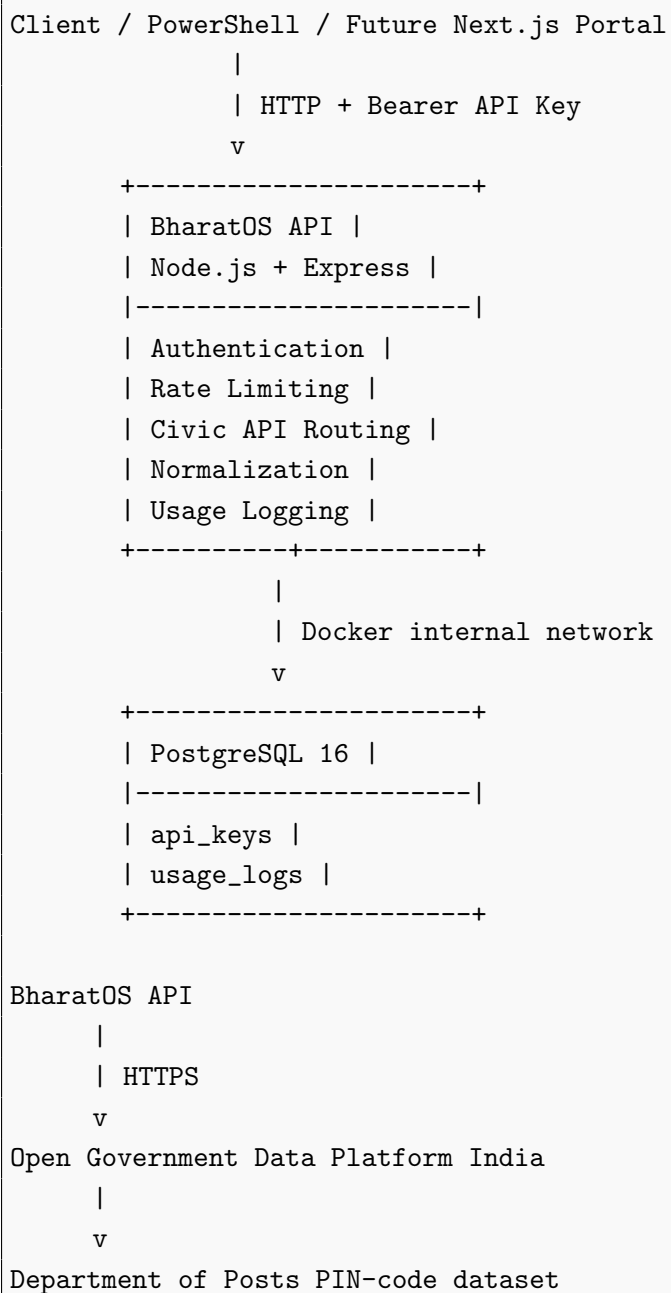
The earlier Python-first phase validated interface design, typed contracts, provenance, scheme discovery, UML, and testability. The current phase concentrates on backend infrastructure and live civic-data integration.

The current development order has been:

1. Verify Docker runtime.
2. Build a minimal Node/Express health endpoint.
3. Add PostgreSQL through Docker Compose.
4. Verify API-to-database connectivity.
5. Add API-key generation and SHA-256 hashing.
6. Add Bearer authentication.
7. Connect a live OGD PIN-code source.
8. Normalize upstream data.
9. Add persistent request logging.
10. Add usage analytics.
11. Add per-key rate limiting.
12. Prepare for frontend containerization and AWS deployment.

3.2 High-Level Architecture

The current working architecture is:



The API container is mapped from container port 8000 to host port 8001. PostgreSQL is not published to a host port and is accessible to the API through the Docker service hostname `postgres:5432`.

3.3 Target Architecture from Earlier Design

The earlier project report described a broader target architecture with browser experiences, an API gateway, authentication/key management, a canonical scheme/location service, PostgreSQL, Redis, request logging, source adapters, scheduled ingestion, and an administrator dashboard.

That target remains useful as a roadmap, but the current implementation status has changed:

- PostgreSQL is now implemented.
- secure random API-key generation is now implemented.
- hashed key storage is now implemented.
- persistent request logging is now implemented.
- per-key rate limiting is implemented in memory.
- Redis remains planned.
- scheduled ingestion remains planned.
- broader scheme/location source adapters remain planned.

3.4 Domain Model and Class Design

The earlier domain design centred on a **NormalizedRecord** abstraction with provenance. Scheme and Location records inherit shared identity and metadata behaviour. **SourceMetadata** is conceptually composed into normalized records because a record without provenance violates the platform contract.

The current database-backed implementation concretely realizes two infrastructure entities:

- **ApiKey**: identifier, hash, prefix, active status, creation time, and last-used time.
- **ApiRequest/UsageLog**: key identifier, endpoint, HTTP method, status code, latency, and timestamp.

Future expansion can map these infrastructure entities into the larger class model from the earlier design.

3.5 Protected API Request Flow

The current request path is:

1. Client sends an HTTP request with `Authorization: Bearer <key>`.
2. BharatOS checks whether the header exists.
3. The supplied key is SHA-256 hashed.
4. PostgreSQL is queried for the matching hash.
5. Invalid or inactive keys return a standardized 401 response.
6. The rate limiter checks the API-key quota.
7. Requests above 20 per minute return HTTP 429.
8. Allowed civic requests call the OGD adapter.
9. The upstream response is normalized.
10. BharatOS returns a stable JSON structure with source metadata.
11. A request log is inserted into PostgreSQL.

The earlier sequence design also included Redis cache-hit/cache-miss branches. These remain future work.

3.6 Ingestion Activity

The earlier report defined the intended ingestion flow as:

```
Collect source
|
Parse native format
|
Validate records
|---- invalid -> quarantine with reason
|
Normalize
|
Deduplicate
|
Enrich provenance
|
Quality threshold
```

```

|---- fail -> rollback/review
|
Commit
|
Index and publish

```

The current live PIN-code implementation does not yet run a scheduled batch-ingestion pipeline. Instead, it performs request-time retrieval and normalization from the OGD API. The ingestion model remains the intended architecture for larger future datasets.

3.7 Technical Stack Evolution

Layer	Earlier Baseline	Current Prototype
Runtime	Python 3.12+	Node.js 24
Web/API	FastAPI	Express.js
Views	Jinja	Next.js planned; current backend tested through HTTP clients
Validation	Pydantic	Explicit route validation and normalization functions
Database	Deferred	PostgreSQL 16 implemented
API Keys	Fixed demo key	Random <code>bos_test_*</code> keys, SHA-256 hashed
Rate Limiting	Deferred	20 requests/minute/key in memory
Usage Logging	Browser-only preview	Persistent PostgreSQL logs
External Data	Static prototype records	Live OGD PIN-code API
Containerization	Planned	Docker Compose implemented
Redis	Planned	Still planned
Cloud	Planned	AWS deployment planned

3.8 Standard API Contract

BharatOS uses versioned API paths such as:

```
GET /api/v1/locations/pincode/110001
```

Protected requests use:

```
Authorization: Bearer bos_test_<generated-value>
```

A successful normalized response includes `data` and `meta`. For example:

```
{
  "data": {
    "pincode": "110001",
    "state": {
      "name": "Delhi"
    },
    "district": {
      "name": "New Delhi"
    }
  },
  "meta": {
    "source": {
      "provider": "Department of Posts",
      "platform": "Open Government Data Platform India"
    },
    "api_version": "v1",
    "request_id": "req_..."
  }
}
```

Standardized errors include a stable code and message. Examples include:

- MISSING_API_KEY
- INVALID_API_KEY
- REVOKED_API_KEY
- INVALID_PINCODE
- PINCODE_NOT_FOUND
- RATE_LIMIT_EXCEEDED
- UPSTREAM_RATE_LIMITED
- UPSTREAM_DATA_SOURCE_ERROR

Implementation

4.1 Earlier Python-first Phase 1 Baseline

The earlier Phase 1 report described a responsive citizen portal, scheme-detail pages, a developer console, API documentation, an administrator preview, and three versioned endpoints. It used FastAPI, Jinja, Pydantic, custom CSS, and minimal JavaScript.

That baseline used eight manually curated scheme snapshots and three PIN-code records. The fixed demonstration key was intentionally labelled as non-production. Nineteen automated tests were reported as passing, and desktop/mobile browser checks verified the primary flows.

This earlier implementation is important because it established the broader product vision and design artefacts, but it should not be confused with the current infrastructure prototype.

4.2 Current Backend File Constraint

The current backend intentionally remains small. The designed backend uses only five principal files:

```
backend/  
|-- package.json  
|-- server.js  
|-- db.js  
|-- ogd.js  
'-- schema.sql
```

The compact structure keeps the prototype understandable while still supporting database persistence, security, live data, analytics, and quotas.

4.3 API-Key Generation and Storage

API keys are generated with cryptographically secure random bytes and prefixed with `bos_test_`. A generated key resembles:

```
bos_test_7af1085ea44ed1ce...
```

Only the SHA-256 hash is stored in PostgreSQL. A short prefix is stored for display and identification.

The plaintext key is returned once during generation. Database inspection confirmed that the original key itself is not stored.

4.4 PostgreSQL Schema

The current schema contains two tables.

4.4.1 `api_keys`

Stores:

- key identifier;
- SHA-256 hash;
- short key prefix;
- active status;
- creation timestamp;
- last-used timestamp.

4.4.2 `usage_logs`

Stores:

- API-key identifier;
- endpoint;
- HTTP method;
- response status;
- latency in milliseconds;
- timestamp.

4.5 Civic-Data Adapter

The OGD adapter communicates with the Department of Posts PIN-code resource using the configured resource endpoint and API key.

For a request such as:

```
GET /api/v1/locations/pincode/110001
```

the current prototype successfully returned real data including:

- PIN code 110001;
- state Delhi;
- district New Delhi;
- postal circle Delhi Circle;
- postal division New Delhi Central Division;
- multiple post-office records such as New Delhi GPO, Connaught Place SO, Patiala House SO, and others.

The adapter also revealed upstream quality irregularities such as `Divreportingcircle` and delivery values that could not always be mapped safely. BharatOS preserves uncertainty as `null` rather than inventing a value.

4.6 Usage Analytics

The protected endpoint:

```
GET /api/usage
```

returns:

- total requests;
- successful requests;
- failed requests;
- average latency;
- recent request records.

One observed result was:

```
{
  "summary": {
    "total_requests": 1,
    "successful_requests": 1,
    "failed_requests": 0,
    "average_latency_ms": 856
  }
}
```

This was an observed single-key state during development and is not claimed as a full platform performance benchmark.

4.7 Rate Limiting

The current prototype uses a fixed-window in-memory rate limiter:

20 requests per minute per API key

The server also returns headers for limit, remaining quota, and reset time.

When the quota is exceeded, the API returns HTTP 429:

```
{
  "error": {
    "code": "RATE_LIMIT_EXCEEDED",
    "message": "API request limit exceeded.",
    "retry_after": 53
  }
}
```

This implementation is intentionally temporary. Its state resets when the API process restarts and is not shared across multiple replicas. Redis is planned for the distributed version.

4.8 Upstream Rate-Limit Handling

During development, the OGD API returned HTTP 429. BharatOS was updated to recognize this upstream condition separately from ordinary failures.

The standardized BharatOS response uses:

```
{
  "error": {
```

```
"code": "UPSTREAM_RATE_LIMITED",
"message":
  "The public data source is temporarily rate limited.
  Please try again shortly."
}
```

This demonstrates the role of BharatOS as an operational abstraction layer, not merely a pass-through proxy.

4.9 Docker Compose Environment

The working local stack consists of:

- **bharatos-api**: Node.js/Express;
- **bharatos-postgres**: PostgreSQL 16.

Observed container state:

```
bharatos-api
0.0.0.0:8001 -> 8000/tcp

bharatos-postgres
5432/tcp
healthy
```

PostgreSQL is intentionally not mapped to a Windows host port. The backend communicates with it through the Docker service name **postgres**.

A named volume persists the database across container restarts.

Results and Evaluation

5.1 Testing Methodology

The current infrastructure prototype has been validated primarily using functional and integration tests through PowerShell, Docker, and PostgreSQL inspection.

- **Health testing:** `/health` verifies API and database connectivity.
- **Authentication testing:** valid, missing, and fake API keys were tested.
- **Database verification:** `psql` was used to inspect stored key hashes and usage records.
- **External integration testing:** real OGD PIN-code requests were executed.
- **Failure testing:** upstream HTTP 429 behaviour was observed and handled.
- **Analytics testing:** logged requests were returned through `/api/usage`.
- **Rate-limit testing:** 25 consecutive protected requests were executed with a fresh key.

The earlier Python-first baseline separately reported nineteen automated tests, static analysis, desktop/mobile visual checks, and five UML output pairs. Those results refer to the earlier baseline and are not presented here as tests of the current Node.js implementation.

5.2 Measured Current Results

Acceptance Check	Expected	Observed
------------------	----------	----------

API health	API + PostgreSQL available	PASS
API-key generation	Random BharatOS key returned	PASS
Key storage	Hash/prefix only	PASS
Valid key	HTTP 200	PASS
Missing key	HTTP 401	PASS
Fake key	HTTP 401	PASS
PIN 110001 lookup	Normalized Delhi/New Delhi data	PASS
Provenance metadata	Source/provider present	PASS
Usage logging	Endpoint/status/latency persisted	PASS
Usage analytics	Summary + recent requests	PASS
Upstream throttling	Standardized BharatOS error	PASS
Requests 1–20 in one window	HTTP 200	PASS
Request 21 onward	HTTP 429	PASS
PostgreSQL health check	Healthy Docker service	PASS

5.3 Rate-Limit Verification

A fresh API key was used to send 25 requests to the protected test endpoint.

Observed result:

```

1 -> 200
2 -> 200
...
19 -> 200
20 -> 200
21 -> 429
22 -> 429
23 -> 429
24 -> 429
25 -> 429

```

The rejected responses included a `retry_after` value, confirming that the prototype quota mechanism functions as intended.

5.4 Performance Observation

One logged live PIN-code request produced:

- endpoint: `/api/v1/locations/pincode/110001`;
- method: GET;
- status: 200;
- latency: 856 ms.

This is a single observed request that includes upstream-network latency. It is not a p95, p99, throughput, or production-availability claim.

5.5 Objective Evaluation

The current prototype meets the essential developer-platform objectives:

- real civic data can be retrieved through a BharatOS endpoint;
- developers authenticate using generated keys;
- keys are not stored in plaintext;
- responses are normalized and source-attributed;
- usage is logged and summarized;
- quotas are enforced;
- the stack is containerized and portable.

The main remaining prototype milestone is the developer-facing frontend, followed by deployment of the same containerized architecture to AWS.

5.6 Risks and Mitigations

Risk	Impact	Mitigation
Stale civic data	Misleading application output	Preserve verification/freshness metadata and authoritative links.

Restricted source use	Legal/trust harm	Use only permitted public/open sources and record provenance/licensing.
Credential theft	API misuse	Hash keys, add HTTPS, revocation, quotas, and future rotation.
Heterogeneous invalid data	Incorrect normalized records	Validate explicitly; return null/unknown where confidence is insufficient.
Source outage	Failed live request	Standardize upstream failures; future caching can retain labelled stale data.
Upstream rate limits	Intermittent unavailability	Detect 429, expose standardized retry behaviour, add caching later.
In-memory quota state	Resets and does not scale across replicas	Migrate to Redis.
Single-container database in early deployment	Lower operational resilience	Move PostgreSQL to RDS when appropriate.

Phased Roadmap

6.1 Completed Infrastructure Milestones

The following features, previously planned for later phases in the earlier report, are now implemented in the current prototype:

- PostgreSQL persistence;
- generated API keys;
- SHA-256 key storage;
- protected API routes;
- live PIN-code adapter;
- request logging;
- usage analytics;
- per-key rate limiting;
- Docker Compose.

6.2 Next Phase – Developer Frontend

The next planned milestone is a small Next.js developer portal with a strict file-count constraint. It should allow a developer to:

- generate an API key;
- store it locally in the browser for demo purposes;

- submit a PIN-code request;
- inspect normalized JSON;
- view request counts, success/failure totals, and average latency;
- inspect recent requests.

6.3 AWS Deployment

After local end-to-end validation, the Dockerized stack can be deployed to AWS EC2.

Initial academic deployment:

```
AWS EC2
|
+-- Docker Compose
   |
   +-- frontend container
   +-- API container
   '-- PostgreSQL container
```

A later improvement can replace the PostgreSQL container with Amazon RDS.

6.4 Redis and Caching

Redis should be introduced only after the current local prototype is stable. Planned responsibilities include:

- distributed rate-limit counters;
- response caching;
- temporary session or ephemeral platform state.

6.5 Dataset Expansion

Future adapters may cover:

- government schemes;
- health;
- education;

- employment;
- agriculture;
- transport;
- environment;
- geospatial civic data.

6.6 Operations and Evaluation

Future evaluation should measure:

- average latency;
- p95 and p99 latency;
- requests per second;
- error rate;
- cache-hit ratio;
- database latency;
- source availability;
- developer time-to-first-successful-call.

Conclusion

7.1 Summary of Work

BharatOS has progressed from a broad software-engineering concept and Python-first demonstration into a working Dockerized developer API prototype.

The earlier baseline established the citizen/discovery vision, API-contract principles, provenance requirements, and UML design. The current implementation adds real infrastructure:

- Node.js/Express backend;
- PostgreSQL;
- secure random API keys;
- SHA-256 key hashing;
- Bearer authentication;
- live Department of Posts PIN-code data;
- normalization;
- persistent request logging;
- usage analytics;
- rate limiting;
- Docker Compose.

7.2 Key Findings

- **Normalization remains valuable even when an upstream API already exists.** Developers benefit from consistent field organization, source metadata, error semantics, security, and usage controls.
- **External civic-data services are operational dependencies.** BharatOS must account for source outages and upstream throttling.
- **Data quality cannot be solved by cosmetic string conversion alone.** Values such as `Divreportingcircle` demonstrate the need for explicit validation and canonicalization rules.
- **API-key hashing is straightforward and materially improves prototype security.** Direct database inspection confirmed that plaintext keys need not be stored.
- **Docker Compose provides a practical migration path from local development to AWS.** The API and database remain isolated while preserving portable runtime configuration.
- **Phase boundaries improve report validity.** Earlier mock/static capabilities are kept distinct from newly implemented persistent and live-data capabilities.

7.3 Future Work and Recommendations

1. Complete the Next.js developer portal.
2. Add the frontend as a Docker Compose service.
3. Add API-key revocation and rotation.
4. Replace the in-memory limiter with Redis.
5. Add response caching.
6. Expand metadata with freshness, source identifiers, and licensing.
7. Add additional civic-data adapters.
8. Implement scheduled ingestion and validation.
9. Add OpenAPI/Swagger documentation.
10. Add automated unit, integration, security, and performance tests for the current Node-based implementation.
11. Deploy the containerized stack to AWS EC2.

12. Move PostgreSQL to RDS if the project requires stronger durability and operational isolation.
13. Add HTTPS/TLS and production secret management.

7.4 Final Remarks

The central value of BharatOS is not a claim to replace official systems. Its value is the software-engineering discipline of placing a source-transparent, secure, versioned, and measurable developer layer above permitted civic information.

The current implementation proves that the idea is technically viable at prototype scale. A developer can generate an authenticated key, call a live civic-data endpoint, receive normalized data with provenance, have the request recorded, inspect usage, and encounter deterministic quota behaviour.

Demonstration Script

1. Start the Docker Compose stack.
2. Show that `/health` reports both API and PostgreSQL connectivity.
3. Generate a new `bos_test_*` API key.
4. Show that a request without a key returns 401.
5. Show that a fake key returns `INVALID_API_KEY`.
6. Call `/api/v1/locations/pincode/110001`.
7. Point out normalized state, district, postal region, post offices, and source metadata.
8. Open `/api/usage` and show total requests, success/failure counts, average latency, and recent requests.
9. Use repeated calls to demonstrate 20 successful requests followed by HTTP 429.
10. Explain that Redis, the Next.js frontend, and AWS deployment are the next steps.

Requirements Traceability

ID	Implementation Evidence	Verification
FR-03	Versioned PIN-code route	Live 110001 request
FR-04	Authentication middleware	Missing/fake key tests
FR-05	Random API-key generator	POST <code>/api/keys</code>
FR-06	SHA-256 key hash	PostgreSQL inspection
FR-07	<code>usage_logs</code> table	Logged endpoint/status/latency
FR-08	GET <code>/api/usage</code>	Summary and recent request output
FR-09	In-memory fixed-window limiter	Requests 21–25 returned 429
FR-10	OGD adapter	Department of Posts PIN-code response
FR-14	Dockerized architecture	Docker Compose local stack complete; AWS pending

Current Prototype API Endpoints

Endpoint	Method	Purpose
/	GET	API identification
/health	GET	API and PostgreSQL health
/api/keys	POST	Generate developer API key
/api/v1/test	GET	Validate protected access
/api/v1/locations/pincode/{pincode}	GET	Live normalized PIN-code lookup
/api/usage	GET	Per-key usage analytics

Bibliography

- [1] Digital India Corporation, *myScheme: National Platform for Government Schemes*. <https://www.myscheme.gov.in/>.
- [2] Department of Posts, Government of India, *All India PIN Code Directory*. Open Government Data Platform India. <https://www.data.gov.in/resource/all-india-pincode-directory-till-last-month>.
- [3] Government of India, *Open Government Data Platform India*. <https://www.data.gov.in/>.
- [4] OpenJS Foundation, *Express.js Documentation*. <https://expressjs.com/>.
- [5] PostgreSQL Global Development Group, *PostgreSQL Documentation*. <https://www.postgresql.org/docs/>.
- [6] Docker Inc., *Docker Documentation*. <https://docs.docker.com/>.
- [7] FastAPI Project, *FastAPI Documentation*. <https://fastapi.tiangolo.com/>.
- [8] Pydantic Services Inc., *Pydantic Documentation*. <https://docs.pydantic.dev/>.
- [9] PlantUML Project, *PlantUML Language Reference Guide*. <https://plantuml.com/>.
- [10] Object Management Group, *Unified Modeling Language Version 2.5.1*. <https://www.omg.org/spec/UML/2.5.1/>.